Davante

18

PROGRAMACIÓN

Bases de datos relacionales

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Introducción a las bases de datos 5

/ 3. Sistemas gestores de bases de datos (SGBD) 6

3.1. Software más populares 6

/ 4. Caso práctico 1: "¿Qué tipo de base de datos usar?" 7

/ 5. API JDBC 7

5.1. Driver 8

5.2. Conexión 10

5.3. Gestión de excepciones 11

/ 6. Ejecución de sentencias 12

6.1. Consulta de datos: SELECT 12

6.2. Borrado de datos: DELETE 13

6.3. Inserción de datos: INSERT 14

6.4. Actualización de datos: UPDATE 15

/ 7. Caso práctico 2: "Optimización del acceso a la base de datos" 15

ÍNDICE

/ 8. ResultSet 16

8.1. Tipos de ResulSet 17

/ 9. Resumen y resolución del caso práctico de la unidad 18

/ 10. Bibliografía 19

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer los diferentes modelos de bases de datos.

Instalar un gestor de bases de datos relacionales.

Configurar el driver JDBC para poder acceder a cualquier SGBD.

Crear la conexión entre un programa y el SGBD.

Ejecutar sentencias de un SGBD en una aplicación que funciona como cliente y mostrar el resultado.

/ 1. Introducción y contextualización práctica

En esta unidad vamos a recordar el concepto de base de datos y qué es una base de datos relacional.

Vamos a ver cómo podemos configurar un proyecto de NetBeans para poder tener acceso a una base de datos relacional y qué bibliotecas tenemos que usar para ello.

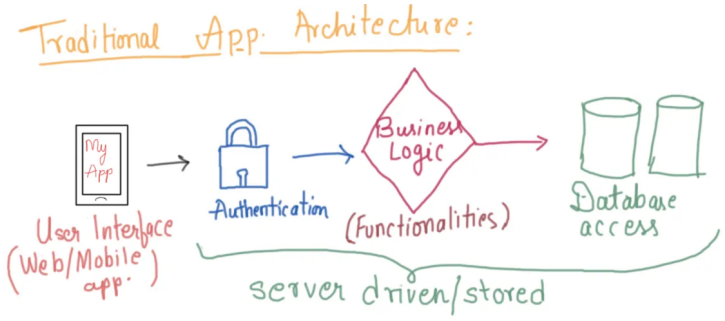


Fig.1. Arquitectura de una aplicación con BD.

También vamos a conocer cómo se realizan las operaciones básicas de inserción, selección, borrado y actualización de datos mediante consultas SQL y la manera de acceder a ellos.

Por último, vamos a ver qué excepciones deberemos manejar a la hora de acceder a una base de datos relacional.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Eficiencia en el acceso a datos"

http://bit.ly/2K2DXK4

/ 2. Introducción a las bases de datos

Las bases de datos nos proveen de un **sistema eficiente para el tratamiento de grandes cantidades de información**. Estas nacieron para mejorar todas las carencias del almacenamiento de información en ficheros. Recordemos de unidades anteriores que los ficheros, cuando se trata de poca información, pueden ser muy útiles, pero cuando contienen grandes cantidades de información, añadiendo su acceso secuencial, no dan una respuesta óptima.

Hay muchos tipos de bases de datos, pero nosotros nos vamos a centrar en las relacionales, que son las que cumplen con el modelo de datos relacional.

¡! Sabías que...

Las bases de datos relacionales tienen sus raíces en la década de 1970, gracias al trabajo pionero de Edgar F. Codd, que publicó un artículo en el que describía un modelo de base de datos basado en las matemáticas de la teoría de conjuntos y la lógica de predicados.

Dentro de las **bases de datos relacionales**, podemos distinguir las siguientes **características comunes**:

- Los **datos** se almacenan en **tablas**, que consisten en filas y columnas. Cada fila representa una entidad única, y cada columna, un atributo de esa entidad.
- Las **tablas** se pueden relacionar entre sí mediante **claves primarias y claves externas**. Las claves primarias son atributos únicos que identifican de manera única cada fila en una tabla, mientras que las claves externas establecen relaciones entre las tablas.
- Las bases de datos relacionales mantienen la **integridad referencial**, lo que significa que las relaciones entre las tablas están controladas y se garantiza que los **datos** sean **consistentes y precisos**.
- Las bases de datos relacionales suelen seguir **reglas de normalización** para evitar la redundancia y mantener la consistencia de los datos. La normalización consiste en dividir las tablas en estructuras más pequeñas y más eficientes.
- **SQL** es el lenguaje estándar utilizado para interactuar con las bases de datos relacionales. Permite realizar consultas, insertar, actualizar y eliminar datos, así como definir y modificar la estructura de la base de datos.
- Las bases de datos relacionales son generalmente **escalables tanto vertical como horizontalmente**, lo que significa que pueden manejar un crecimiento en la cantidad de datos y en la carga de trabajo mediante la adición de recursos o la distribución de datos en múltiples servidores.

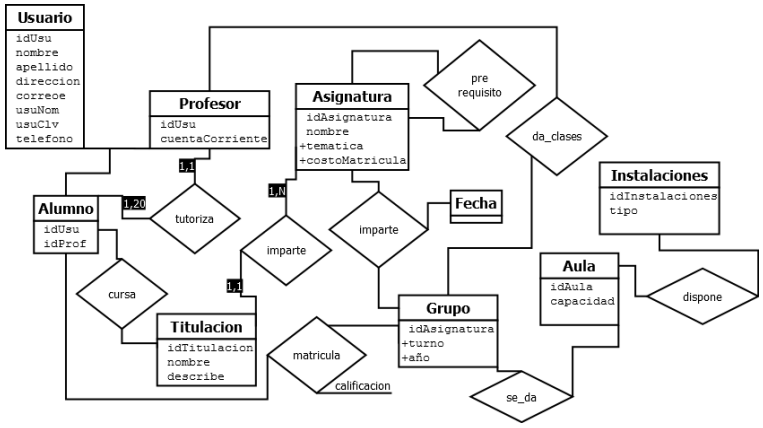


Fig.2. Modelo Entidad-Relación BD.

/ 3. Sistemas gestores de bases de datos (SGBD)

Constituyen una herramienta fundamental en la tecnología de la información por distintas razones. Entre estas, se incluye que los SGDB:

- Proporcionan un medio eficiente y estructurado para almacenar **grandes cantidades de datos de manera organizada**.
- Permiten controlar el acceso a los datos mediante la implementación de **mecanismos de autenticación y autorización**.
- Incluyen mecanismos para garantizar la integridad de los datos, como la aplicación de **restricciones de clave primaria y foránea**, así como **reglas de validación de datos**.
- Permiten que **múltiples usuarios** accedan y actualicen la base de datos de forma concurrente, mientras mantienen la **integridad de los datos**.
- Están diseñados para manejar **grandes volúmenes de datos y cargas de trabajo variables**.
- Ofrecen funcionalidades para realizar **copias de seguridad periódicas y recuperación de los datos**.

3.1. Software más populares

Los *software* más populares para el diseño y la gestión de bases de datos son los siguientes:

- **MySQL**: Es un sistema de gestión de bases de datos relacional de código abierto. Conocido por su velocidad y fiabilidad, es mantenido por Oracle Corporation.
- **Microsoft SQL Server**: Desarrollado por Microsoft, es un sistema de gestión de bases de datos relacional que se ejecuta principalmente en entornos Windows.
- **Oracle Database**: Es un sistema de gestión de bases de datos relacional desarrollado por Oracle. Es conocido por su escalabilidad, seguridad y capacidad para gestionar grandes volúmenes de datos.
- **PostgreSQL**: Se trata de un sistema de gestión de bases de datos relacional de código abierto y de propósito general. Es conocido por su robustez, conformidad con los estándares y amplio conjunto de características avanzadas.
- **SQLite**: Es un motor de base de datos relacional ligero que no requiere un servidor separado y se integra directamente en la aplicación. Es ideal para aplicaciones embebidas, dispositivos móviles y proyectos de desarrollo rápido.
- **MongoDB**: Aunque no es un sistema relacional, es ampliamente utilizado, especialmente en aplicaciones web y móviles. Almacena datos en formato JSON y es conocido por su escalabilidad y flexibilidad en esquemas de datos.



Fig.3. Sistemas gestores de bases de datos.

Audio 1. "¿Qué gestor de bases de datos elegir?"

http://bit.ly/3nzqIOS

/ 4. Caso práctico 1: "¿Qué tipo de base de datos usar?"

Planteamiento: Existen muchos tipos de bases de datos, cada una diseñada para satisfacer diferentes necesidades y casos de uso, por lo que la elección de uno depende de las necesidades específicas del proyecto y las características de los datos que se necesita almacenar y gestionar.

Nudo:

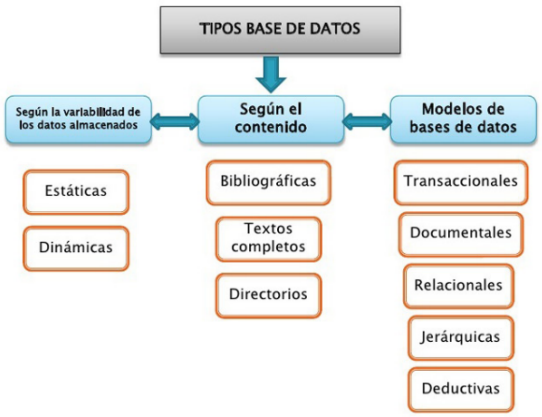


Fig.4. Tipos de bases de datos.

En el caso de tener que realizar una aplicación y teniendo en cuenta la gran cantidad de tipos de bases de datos que existen, ¿qué tipo será mejor a la hora de usarlo en un programa?

Desenlace: Esta pregunta siempre va a estar presente a la hora de realizar un proyecto real. En todo proyecto que se precie, habrá de por medio una base de datos, ya que, con toda probabilidad, el almacenamiento de información, y no precisamente poca, estará presente.

Normalmente, lo que se use va a ser una base de datos relacional, ya que este tipo de bases de datos cubre las necesidades normales de la gran mayoría de programas del mercado, pero si existen otros tipos de bases de datos es por alguna razón. Por ejemplo, las no relacionales basadas en XML o en JSON se usan mucho en sistemas web, donde una base de datos relacional no puede presentar la solución óptima.

/ 5. API JDBC

La API JDBC (Java Database Connectivity) es una **interfaz de programación de aplicaciones estándar de Java** que permite a los desarrolladores **interactuar con bases de datos relacionales utilizando el lenguaje de programación Java**.

Esta biblioteca está incluida dentro del paquete java.sql e incluye un conjunto de clases e interfaces que permiten a los programadores ejecutar consultas SQL, recuperar y manipular datos, y administrar conexiones con la base de datos desde sus aplicaciones Java.

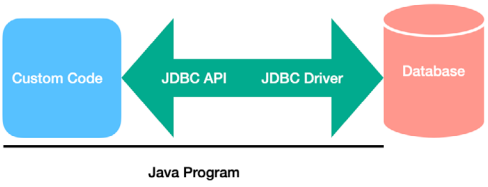


Fig.5. Java JDBC.

Las principales **funcionalidades** son:

- **Establecimiento de conexiones**: La API JDBC permite a los desarrolladores establecer conexiones con una base de datos utilizando diversos métodos de conexión, como conexiones JDBC estándar, conexiones de origen de datos (DataSource) y conexiones de grupo (Connection Pooling).

- **Ejecución de consultas SQL**: Los desarrolladores pueden ejecutar consultas SQL utilizando objetos de tipo **Statement** o **PreparedStatement**. Estos objetos permiten ejecutar consultas de selección, inserción, actualización y eliminación de datos en la base de datos.

- **Recuperación y manipulación de datos**: La API JDBC proporciona métodos para recuperar y manipular datos devueltos por consultas SQL. Los resultados de las consultas se pueden procesar utilizando objetos de tipo **ResultSet**, que permiten iterar a través de las filas devueltas y acceder a los valores de las columnas.

- **Control de transacciones**: La API JDBC ofrece soporte para la gestión de transacciones en bases de datos relacionales. Los desarrolladores pueden controlar el inicio, la finalización y el *rollback* de transacciones utilizando objetos de tipo **Connection**.

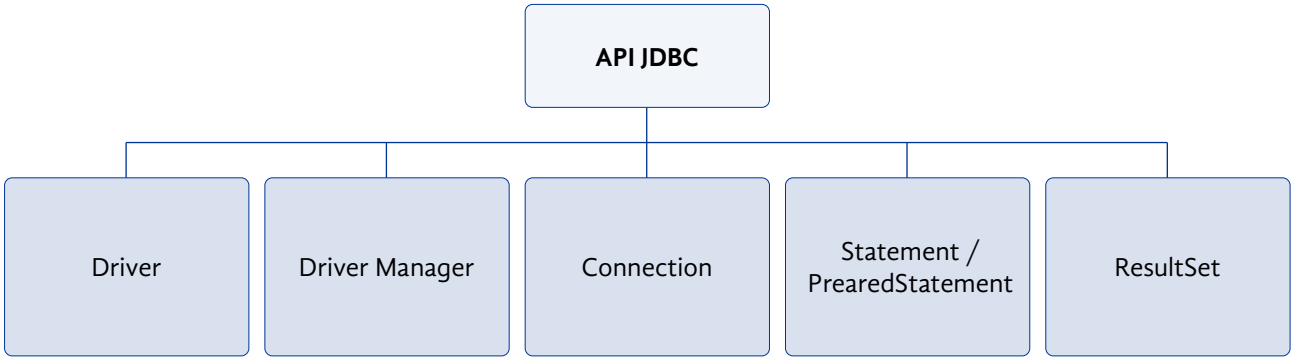


Fig.6. Principales clases JDBC API.

> **¡! Sabías que...**
>
> La capacidad de JDBC para manejar conexiones, ejecutar consultas y procesar resultados de manera eficiente ha hecho que sea una herramienta esencial para el desarrollo de aplicaciones empresariales y de bases de datos en Java.

5.1. Driver

Ya se ha visto que existen distintos SGBD y que cada uno de ellos tiene su propia API y sus propios protocolos de llamadas responsables de la comunicación entre el código Java y la base de datos: MySQL, PotsgreSQL, MongoDB, etc.

La API de JDBC no conoce los detalles propios de cada SGBD, por tanto, la pregunta es obvia: ¿Cómo es posible la comunicación entre los protocolos de la API JDBC y la base de datos?

Existe una **clase de la API JDBC denominada Driver, que es específica para cada SGBD**, de manera que por cada fabricante de un SGBD (figura 7), existe un *driver* específico:

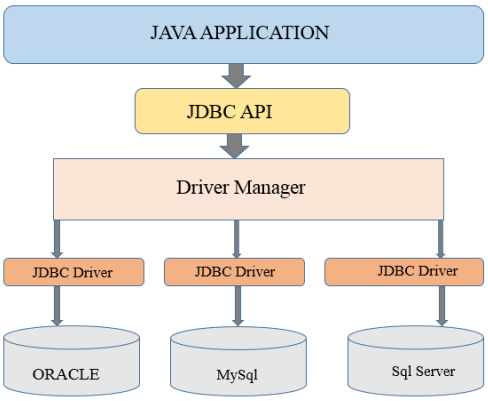


Fig.7. Driver Manager SGBD.

De esta manera, el driver permite a la aplicación Java, a través de la API, emplear los servicios de cualquier SGBD sin necesidad de conocer los detalles de implementación.

Driver preinstalado

Existen algunos drivers que vienen preinstalados, como es el caso de PostgreSQL o MySQL. Pulsamos con el botón derecho a la izquierda de NetBeans, donde pone *Libraries* o Bibliotecas, y seleccionamos Agregar librería, seleccionamos nuestro conector como muestra la siguiente imagen (figura 8):

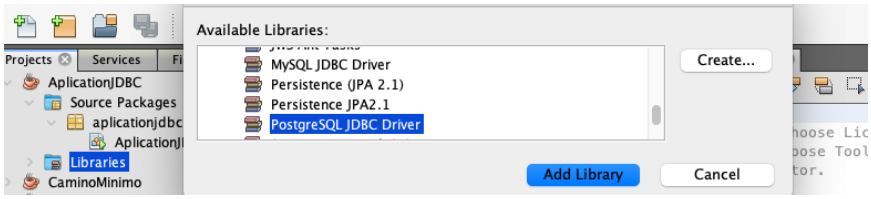


Fig.8. Drivers disponibles en Netbeans.

Añadir un driver

En caso de que el driver del SGBD que queremos emplear no apareciese, debemos descargarlo de la web oficial del fabricante y descargar el fichero '.jar'.

A continuación, tendremos que importarlo como biblioteca en cada uno de nuestros proyectos que lo necesite. Para ello pulsamos con el botón derecho a la izquierda de NetBeans, donde pone *Libraries* o Bibliotecas, y seleccionamos Agregar archivo JAR, seleccionamos nuestro conector y listo.

El conector que acabamos de descargar nos provee de las siguientes clases para la manipulación de información:

- **Connection**: Clase que nos permitirá instanciar objetos donde se almacenará nuestra conexión.
- **Statement**: Clase que nos permitirá instanciar objetos que utilizaremos para realizar las consultas SQL.
- **ResultSet**: Clase que nos permitirá instanciar objetos que nos devolverán los resultados de las consultas SQL.

5.2. Conexión

Una vez añadido el SGBD y su driver específico, es necesario crear una conexión entre la aplicación y el SGBD. Esta conexión se encarga de comunicar ambas partes permitiendo, por tanto, que las sentencias SQL, en forma de Statement o PreparedStatement, viajen en dirección a la BD, y el resultado de la sentencia SQL, ResultSet, lo haga en sentido contrario, como muestra la figura 9:

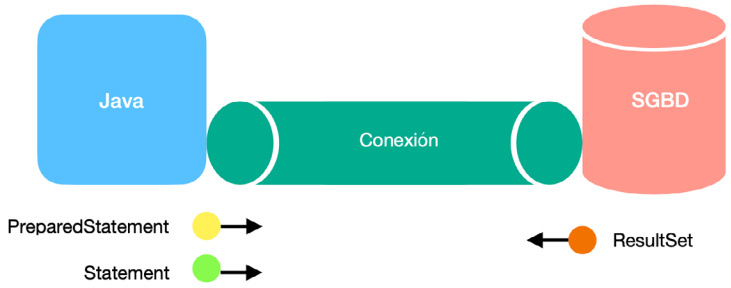


Fig.9. Conexión con el SGBD.

La conexión, ya vimos en la figura 7, que la creábamos a través del DriverManager, por tanto, el primer paso será crear una conexión a través de él. En este caso, el DriverManager tiene el siguiente método estático:

Connection getConnection(String url, String usuario, String password);

Los parámetros que aparecen son los siguientes:

Url indica la base de datos a la que se quiere acceder. En el caso de ser para MySQL, tendrá el siguiente formato:

"jdbc:mysql://<servidor>/<base de datos>"

Donde:

- 'Servidor' indica la dirección IP del servidor donde está instalada la BD, que, en caso de ser en la máquina con la que se trabaja, será localhost.
- 'Base de datos' indica el nombre de la BD.
- Los dos parámetros restantes son una cadena con el nombre de usuario y su contraseña.

Para desconectar de la base de datos, simplemente llamaremos al método close() del objeto de la clase Connection como muestra el código de la siguiente imagen (figura 10):

```
import java.sql.Connection;
import java.sql.DriverManager;

public class ConexionBD {
    // Información de la base de datos
    static final String URL = "jdbc:mysql://localhost:3306/MedacBD";
    static final String USUARIO = "usuario";
    static final String CONTRASEÑA = "contraseña";
    Connection connection;
    String sql;

    public static void main(String[] args) {

        connection = DriverManager.getConnection(URL, USUARIO, CONTRASEÑA);

        connection.close();

    }
}
```

Fig.10. Conexión con MedacBD.

5.3. Gestión de excepciones

Las operaciones con bases de datos son muy propensas a fallar y por ello vamos a tener que utilizar la gestión de excepciones prácticamente con todas las operaciones que hemos visto.

Son varios los motivos por los que podemos tener una excepción en el tratamiento de bases de datos, entre los que destacan los siguientes casos:

- No se ha conectado a la base de datos.
- Los datos de conexión son incorrectos, bien porque la base de datos no existe o porque el usuario o la contraseña son incorrectos.
- La instrucción SQL que se quiere lanzar a la base de datos tiene fallos sintácticos.
- La base de datos se ha corrompido.
- No se tienen permisos para realizar alguna de las operaciones.

El t**ratamiento de excepciones** se realizará mediante un **bloque try-catch**. En el caso de las bases de datos, al igual que pasaba con los ficheros, sería conveniente **cerrar las conexiones** en el **bloque finally**.

El **método getConnection() de la clase Connection** puede provocar dos tipos de **excepciones**:

- **SQLException**: Excepción de nivel superior en JDBC que indica un error relacionado con la base de datos. Puede ser causada por una variedad de problemas, como conexiones perdidas, errores de sintaxis SQL, violaciones de restricciones de integridad, problemas de acceso concurrente, etc.
- **SQLTimeoutException**: Subclase de SQLException que se lanza cuando una operación de base de datos no se completa dentro del tiempo de espera especificado. Esta excepción generalmente ocurre cuando una consulta SQL o una operación de actualización de base de datos tarda más tiempo del permitido en completarse.

El código de la siguiente imagen (figura 11) muestra el tratamiento de excepciones.

```java
import java.sql.SQLException;
import java.sql.Connection;
import java.sql.DriverManager;

public class ConexionBD {
    // Información de la base de datos
    static final String URL = "jdbc:mysql://localhost:3306/MedacBD";
    static final String USUARIO = "usuario";
    static final String CONTRASEÑA = "contraseña";
    Connection connection = null;

    public static void main(String[] args) {

        connection = DriverManager.getConnection(URL, USUARIO, CONTRASEÑA);

        try {
            // Paso 1: Cargar el driver JDBC
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Paso 2: Establecer la conexión
            connection = DriverManager.getConnection(URL, USUARIO, CONTRASEÑA);

            if (connection != null) {
                System.out.println("¡Conexión exitosa!");
                // Aquí puedes realizar operaciones con la base de datos
            } else {
                System.out.println("No se pudo establecer la conexión.");
            }

        } catch (SQLException e) {
            System.out.println("Error al conectar a la base de datos: " + e.getMessage());
        } catch (ClassNotFoundException e) {
            System.out.println("No se encontró el driver JDBC: " + e.getMessage());
        } finally {
            // Paso 3: Cerrar la conexión
            try {
                if (connection != null) {
                    connection.close();
                }
            } catch (SQLException e) {
                System.out.println("Error al cerrar la conexión: " + e.getMessage());
            }
        }
    }
}
```

Fig.11. Tratamiento de excepciones en conexión a BD.

/ 6. Ejecución de sentencias

En el momento en el que queramos realizar una **sentencia SQL**, se deberá crear un **objeto de la clase Statement** mediante la siguiente sentencia:

```
Statement sentencia = connection.createStatement();
```

6.1. Consulta de datos: SELECT

En SQL, para consultar datos de una tabla, se utiliza el **comando SELECT**, al cual le pasaremos las columnas que queremos y una serie de **restricciones** a las mismas mediante las **instrucciones WHERE, GROUP BY, ORDER BY**, etc. Por ejemplo:

```
SELECT * FROM persona WHERE edad >= 18
```

En nuestro caso vamos a usar la misma sentencia SELECT, solo que tendremos que almacenarla en un objeto de tipo String (sql).

Aquí, al igual que ocurre con las demás sentencias, hay que tener un extremo **cuidado** con el uso de las **comillas simples** que preceden a los valores del tipo cadena y recordar que, cuando no se trate de una cadena, sino de un número tanto entero como real, llevará dichas comillas simples.

Para ejecutar las sentencias SELECT, creamos la sentencia mediante Statement:

```
Statement sentencia = connection.createStatement();
```

A continuación se ejecuta la consulta mediante el método executeQuery(String sql), al cual le pasaremos como un String la consulta SELECT que deseamos ejecutar. Este método devolverá un objeto de tipo ResultSet, que contendrá el resultado de la ejecución de la consulta:

```
ResultSet rset = sentencia.executeQuery(sql_select);
```

Este método es susceptible de lanzar una excepción SQLException, por lo que tendremos que tratarla con un bloque try-catch.

El código de la siguiente imagen (figura 12) muestra la sentencia SELECT y el tratamiento de excepciones.

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ConsultaSelect {
    static final String URL = "jdbc:mysql://localhost:3306/MedacBD";
    static final String USUARIO = "root";
    static final String CONTRASEÑA = "medac";

    public static void main(String[] args) {
        Connection connection = null;
        Statement statement = null;
        ResultSet resultSet = null;

        try {
            // Paso 1: Establecer conexión
            connection = DriverManager.getConnection(URL, USUARIO, CONTRASEÑA);

            // Paso 2: Crear objeto Statement
            statement = connection.createStatement();

            // Paso 3: Ejecutar la consulta SQL
            String sql = "SELECT * FROM Alumnos";
            resultSet = statement.executeQuery(sql);

            // Paso 4: Procesar los resultados
            while (resultSet.next()) {
                // Obtener valores de las columnas por nombre
                String nombre = resultSet.getString("nombre");
                // Procesar los valores obtenidos
                System.out.println("Nombre: " + nombre);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Cerrar recursos
            try {
                if (resultSet != null) {
                    resultSet.close();
                }
                if (statement != null) {
                    statement.close();
                }
                if (conexion != null) {
                    conexion.close();
                }
            } catch (SQLException e) {
                System.out.println(e);
            }
        }
    }
}
```

Fig.12. Sentencia SELECT.

Vídeo 1. "Ejecución de una consulta SELECT"

https://bit.ly/2XB27yl

6.2. Borrado de datos: DELETE

En SQL, para borrar datos de una tabla, se utiliza el **comando DELETE**, al cual le pasaremos el nombre de la tabla de la que queremos eliminar los datos y una serie de **restricciones** con la **instrucción WHERE**. Por ejemplo:

***DELETE FROM* persona *WHERE* edad >= '18'**

En nuestro caso, vamos a usar la misma sentencia DELETE, solo que tendremos que almacenarla en un objeto del tipo String.

Es importante que las sentencias DELETE lleven su WHERE correspondiente, ya que, de no ser así, borrarán todos los datos de la tabla que se les diga.

Aquí hay que tener un extremo **cuidado** con el uso de las **comillas simples** que preceden a los valores del tipo cadena y recordar que cuando no se trate de una cadena, sino de un número tanto entero como real, llevará dichas comillas simples.

También se puede emplear las **comillas dobles** mediante el uso de la secuencia 'escape'. En el ejemplo previo la sentencia se podría escribir de cualquiera de las dos maneras:

```
String consulta = "DELETE FROM persona WHERE edad >= '18'"

String consulta = "DELETE FROM persona WHERE edad >= \"18\""
```

Las comillas en rojo marcan las empleadas en la cadena JAVA y el resto, las empleadas por la sentencia SQL.

Para ejecutar las sentencias DELETE vamos a seguir el siguiente código:

```
Statement stmt = connection.createStatement();
int filas_afectadas = stmt.executeUpdate(consulta_delete);
```

El primer paso es crear un objeto de la clase Statement, el cual va a obtener la base de datos para poder realizar una acción sobre ella. Ahora, una vez hecho esto, sobre el objeto creado, utilizaremos el método executeUpdate, al cual le pasaremos como un String la consulta DELETE que deseamos ejecutar.

Este método devolverá un entero, que representará el número de filas afectadas por la sentencia ejecutada, es decir, la cantidad de filas que han sido eliminadas con la consulta.

Este método lanzará una excepción SQLException, por lo que tendremos que tratarla con un bloque try-catch, como se muestra en el código de la siguiente imagen (figura 13).

```java
int filas_afectadas = -1;
String consulta = "DELETE FROM pelicula WHERE titulo='Los Goonies'";

try {
    Statement stmt = connection.createStatement();
    filas_afectadas = stmt.executeUpdate(consulta);
    System.out.println(filas_afectadas + " fila(s) insertada(s).");
} catch (SQLException ex) {
    System.out.println("ERROR: " + ex.getMessage());
}
```

Fig.13. Sentencia Delete.

6.3. Inserción de datos: INSERT

En SQL, para insertar datos en una tabla, se utiliza el **comando INSERT**, al cual le pasaremos el nombre de la tabla, los nombres de las columnas y los valores a insertar. Vamos a realizar un ejemplo:

```
INSERT INTO Alumno (DNI, nombre, apellidos) VALUES ('147A', 'Pepe', 'López')
```

En nuestro caso, vamos a usar la misma sentencia INSERT, solo que tendremos que almacenarla en un objeto del tipo String.

Aquí, al igual que ocurre con las demás sentencias, hay que tener un extremo **cuidado** con el uso de las **comillas simples** que preceden a los valores del tipo cadena y recordar que, cuando no se trate de una cadena, sino de un número tanto entero como real, llevará dichas comillas simples.

Para ejecutar las sentencias INSERT vamos a seguir el siguiente código:

```
Statement stmt = connection.createStatement();
int filas_afectadas = stmt.executeUpdate(consulta_insert);
```

El primer paso es crear un objeto de la clase Statement, el cual va a obtener la base de datos para poder realizar una acción sobre ella. Una vez hecho esto, sobre el objeto creado, utilizaremos el método executeUpdate, al cual le pasaremos como un String la consulta INSERT que deseamos ejecutar. Este método devolverá un entero, que representará el número de filas afectadas por la sentencia ejecutada, en este caso siempre va a ser una, ya que insertamos una fila nueva.

Este método lanzará una excepción SQLException, por lo que tendremos que tratarla con un bloque try-catch.

6.4. Actualización de datos: UPDATE

En SQL, para actualizar datos de una tabla, se utiliza el **comando UPDATE**, al que le pasaremos el nombre de la tabla, los datos a modificar y una serie de **restricciones** con la **instrucción WHERE**. Por ejemplo:

```
UPDATE persona SET edad = '20' WHERE nombre LIKE F%
```

Esta sentencia se emplea de manera análoga a como se usan las sentencias INSERT y DELETE.

El código de la siguiente imagen (figura 14) muestra la sentencia UPDATE:

```
int filas_afectadas = -1;
String consulta = "UPDATE pelicula SET año='1982' WHERE titulo='Blade Runner'";

try {
    Statement stmt = connection.createStatement();
    filas_afectadas = stmt.executeUpdate(consulta);
    System.out.println(filas_afectadas + " fila(s) insertada(s).");
} catch (SQLException ex) {
    System.out.println("ERROR: " + ex.getMessage());
}
```

Fig.14. Sentencia UPDATE.

/ 7. Caso práctico 2: "Optimización del acceso a la base de datos"

Planteamiento: Organizar el acceso a la base de datos implica varias capas y patrones de diseño para asegurar que el código sea mantenible, reutilizable y fácil de probar.

Nudo:

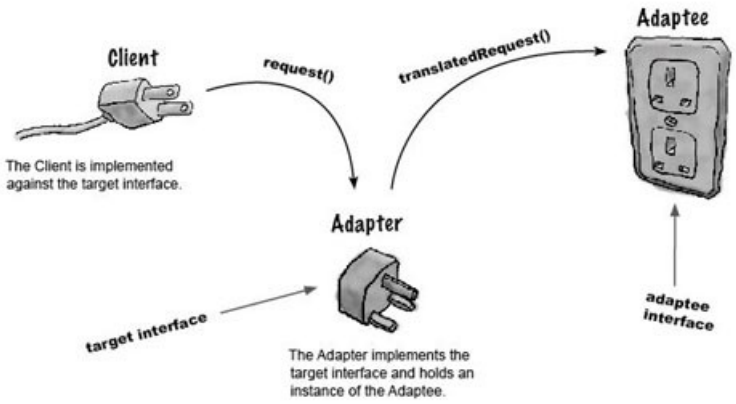


Fig.15. Tipos de bases de datos.

A la hora de realizar consultas a la base de datos, existe mucho código que se repite a lo largo del programa cada vez que se realiza la consulta. ¿Crees que se está realizando de la manera correcta repitiendo el código en cada consulta?

Desenlace: Ya se ha visto en repetidas ocasiones a lo largo de la asignatura que una de las grandes ventajas de la programación dirigida a objetos es la reutilización de código. A la hora de conectar a la base de datos esto no va a ser diferente, y deberemos repetir lo mínimo el código que escribamos, para así poder hacer los programas más estructurados.

En el caso del acceso a las bases de datos, esto se puede realizar de una forma bastante fácil creando una clase para tal efecto. Es decir, podremos crear una clase que nos gestione todo lo que tenga que ver con el acceso a la base de datos, con ella podremos conectarnos, desconectarnos, realizar consultas, inserciones, etc.

/ 8. ResultSet

La clase ResultSet en Java es una interfaz que **proporciona métodos para iterar sobre un conjunto de resultados obtenidos de una consulta SQL**. Es una parte fundamental de JDBC (Java Database Connectivity) y se utiliza para procesar y manipular los datos devueltos (con una sentencia get por cada tipo de datos) por una consulta SELECT.

Imaginemos que realizamos la siguiente consulta a la base de datos:

```
String sql = SELECT num, nombre, media FROM Alumnos WHERE fnac > '1990-12-21';
```

Se trata de una consulta a la base de datos de Alumnos en la que se pide el número, nombre y nota media de los alumnos que hayan nacido en la fecha posterior al 21 de diciembre de 1990, obteniendo los resultados mostrados en la tabla.

num	nombre	media
1	Jorge Rodríguez	9,3
2	Francisco José García	7,6
3	Francisco Jiménez	8,9

Tabla 1. Listado de alumnos resultante.

En un primer momento, el cursor no apuntará a ninguna fila y se situará justo antes de la primera fila, como se muestra en la siguiente tabla 2:



num	nombre	media
1	Jorge Rodríguez	9,3
2	Francisco José García	7,6
3	Francisco Jiménez	8,9

Tabla 2. Cursor inicial.

Por lo tanto, para poder avanzar y realizar una primera lectura, es necesario colocar el cursor en la primera fila. Para ello se emplea el método **boolean next()**. Este método se encarga de mover el cursor a la siguiente fila indicando 'false' si termina de recorrer la tabla o 'true' en caso contrario.

La estructura empleada para recorrer la tabla de resultados sería:

```
ResultSet rs= sentencia.executeQuery(sql);
while(rs.next()){
        ...//Extraer datos de la fila activa
}
```

Una vez que pase por el bucle y realice una primera lectura, el cursor se pondrá en la Fila 1 (tabla 3) y podrá extraer los datos que necesite.



num	nombre	media
1	Jorge Rodríguez	9,3
2	Francisco José García	7,6
3	Francisco Jiménez	8,9

Tabla 3. Cursor tras hacer una lectura.

8.1. Tipos de ResulSet

Hasta ahora, los objetos ResultSet son de solo lectura y el cursor solo avanza hacia delante. Sin embargo, existen varios tipos de ResultSet para manejar diferentes situaciones y requisitos.

Para ello, es necesario crear otro tipo de Statement mediante un método sobrecargado de la clase Connection:

```
createStatement(int tipo ResultSet, int concurrencia);
```

El tipo ResultSet puede ser de tres tipos:

- **TYPE_FORWARD_ONLY**: Solo permite el movimiento hacia delante a través de los resultados, de manera secuencial, sin la posibilidad de retroceder. Es el tipo de ResultSet predeterminado cuando se crea un Statement y el más eficiente en términos de recursos, ya que no necesita mantener un búfer grande en memoria.

```
Statement statement = connection.createStatement(ResultSet.TYPE_FORWARD_ONLY, ResultSet.CONCUR_READ_ONLY);
ResultSet resultSet = statement.executeQuery("SELECT * FROM tabla");
```

Fig.16. ResultSet TYPE_FORWARD_ONLY.

- **TYPE_SCROLL_INSENSITIVE**: Permite moverse hacia delante y hacia atrás a través de los resultados, y también posicionarse en cualquier fila específica. Los cambios realizados en la base de datos no se reflejan en el ResultSet, es decir, es insensible a los cambios externos. Es menos eficiente en términos de recursos que el tipo TYPE_FORWARD_ONLY.

```
Statement statement = connection.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_READ_ONLY);
ResultSet resultSet = statement.executeQuery("SELECT * FROM tabla");
```

Fig.17. ResultSet TYPE_ SCROLL_INSENSITIVE.

- **TYPE_SCROLL_SENSITIVE**: Es similar al tipo TYPE_SCROLL_INSENSITIVE. Permite moverse hacia delante y hacia atrás a través de los resultados y posicionarse en cualquier fila específica. A diferencia del tipo TYPE_ SCROLL_INSENSITIVE, este tipo de ResultSet es sensible a los cambios externos en la base de datos, lo que significa que los cambios realizados por otras conexiones se reflejan en el ResultSet. Es menos común y puede ser menos eficiente en términos de rendimiento que los otros tipos de ResultSet.

```
Statement statement = connection.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
ResultSet resultSet = statement.executeQuery("SELECT * FROM tabla");
```

Fig.18. ResultSet TYPE_SCROLL_SENSITIVE.

Además de estos tipos, JDBC también proporciona opciones de concurrencia para ResultSet:

- **CONCUR_READ_ONLY**: Los datos que contiene el ResultSet son de solo lectura.
- **CONCUR_UPDATABLE**: Los datos que contiene el ResultSet permiten actualizaciones.

¡! Sabías que...

Es importante seleccionar el tipo de ResultSet adecuado según las necesidades específicas de tu aplicación, considerando aspectos como el rendimiento, la sensibilidad a los cambios externos y los recursos disponibles.

/ 9. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema se ha dado un pequeño repaso a lo que son las bases de datos y las ventajas que nos proveen. En concreto, nos hemos centrado en las bases de datos relacionales, donde el acceso a la información se puede realizar de una forma muy sencilla mediante la biblioteca que permite acceder a ella.

También se ha visto cómo podemos recuperar información de la base de datos, así como insertarla, borrarla y modificarla, para poder cubrir todo el abanico de necesidades básicas, teniendo en cuenta la gestión eficiente de las excepciones que se pueden producir en el acceso a una base de datos.

A continuación podrás ver el esquema resumen del tema.

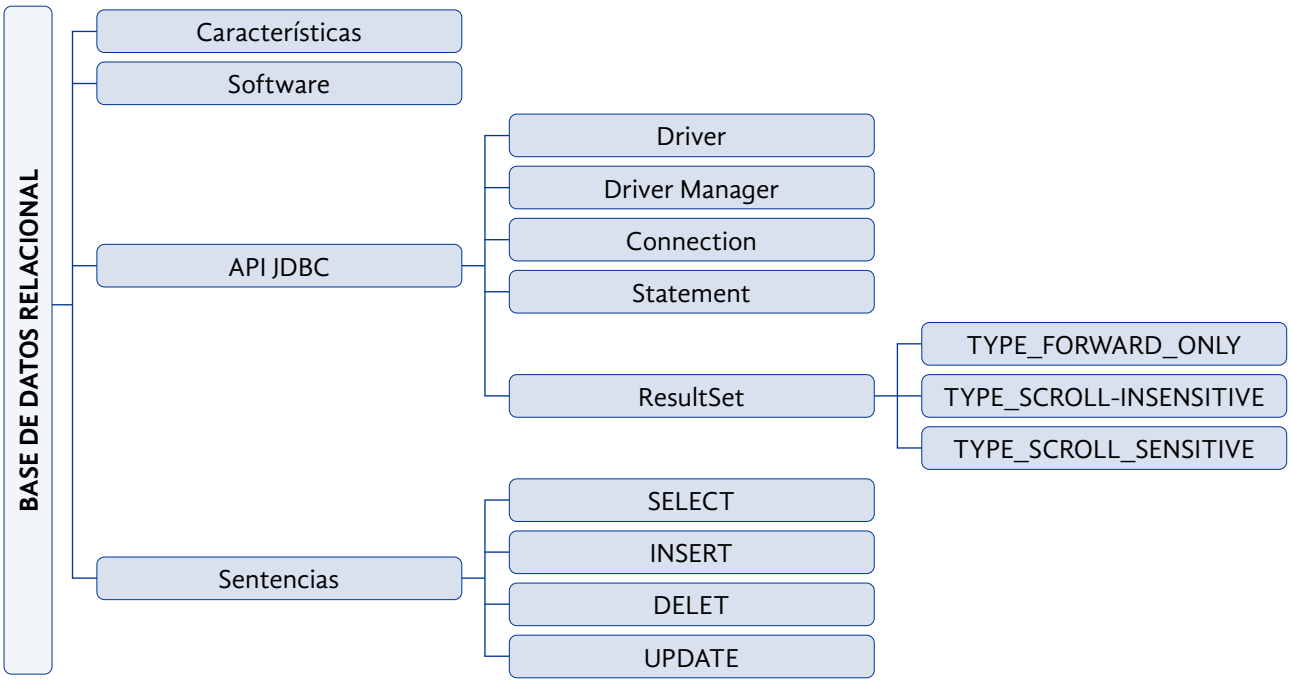


Fig.19. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Cualquier programa que se considere profesional deberá poder dar un acceso eficiente a la información.

En este caso práctico, se debe poder acceder a una información que puede ser bastante grande, ¿cuántos animales pueden pasar por una clínica veterinaria si esta es lo suficientemente grande? La respuesta es 'muchos', y solo con eso ya se deberá tratar un volumen considerable de información.

Ya se ha visto, en unidades pasadas, que se pueden emplear ficheros para el almacenamiento de información, pero estos son muy lentos, además de secuenciales. La solución a todos estos problemas está en el uso de una base de datos, ya que estas están totalmente optimizadas para almacenar mucha información y acceder de forma rápida a ella.

/ 10. Bibliografía

Bhargava, A. Y. (2019). *Algoritmos: Una guía ilustrada para programadores y curiosos*. Madrid: Ediciones Anaya Multimedia (Grupo Anaya).

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Knuth, D. E. (1997). *The Art of Computer Programming* (Vol. 1: *Fundamental Algorithms*) (3.ª ed.). Addison-Wesley.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.